

SHIVA SAI PERSEVERANCE

1. Purpose of the IT Industry

The IT Industry exists to solve business and human problems using technology.

Examples:

- Banking → Online transactions
- Healthcare → Patient records
- E-commerce → Online shopping
- Education → Learning platforms

2. Product Delivery

A product is built once and sold to many customers.

Examples:

- Windows
- Microsoft Office
- Photoshop
- Zoom

3. Service Delivery

A service is customized for a specific client.

Examples:

- TCS
- Infosys
- Wipro
- Accenture

4. Human-Level Language

High-level languages are designed to be read, written, and understood by humans.

In computers and programming, this refers to high-level languages.

Examples

- Python
- Java
- C
- JavaScript

5. Machine-Level Language

Machine language is a low-level programming language that is understood by computers. Machine language is made up of binary bits 0 and 1.

Example:

10101010 11001100 00001111

A computer can understand this directly, but humans cannot easily understand it.

6. Binary code

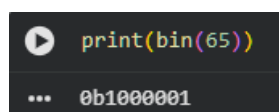
A **binary code** is the value of a Data encoding convention represented in a binary notation that usually is a sequence of 0s and 1s, sometimes called a bit *string*.

Example:

Character A

ASCII → 65

Binary → 01000001



7. Data Centre

A data centre is a centralised physical facility that houses networked computers, servers, and data storage drives.

Examples:

- Google Data Centres
- AWS Data Centres
- Azure Data Centres

8. Server

A **server** is a computer that provides data, services, or resources to other computers (called clients) over a network.

Example:

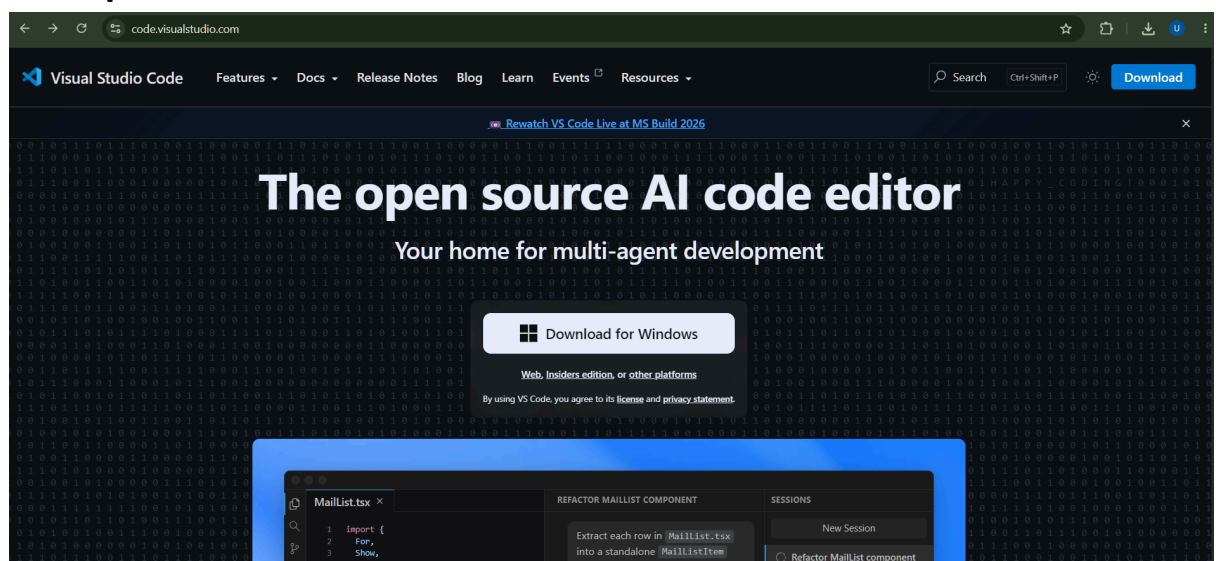
Imagine a restaurant:

- **Customer** → asks for food
- **Server (waiter)** → brings food

9. IDE Installation (VS Code)

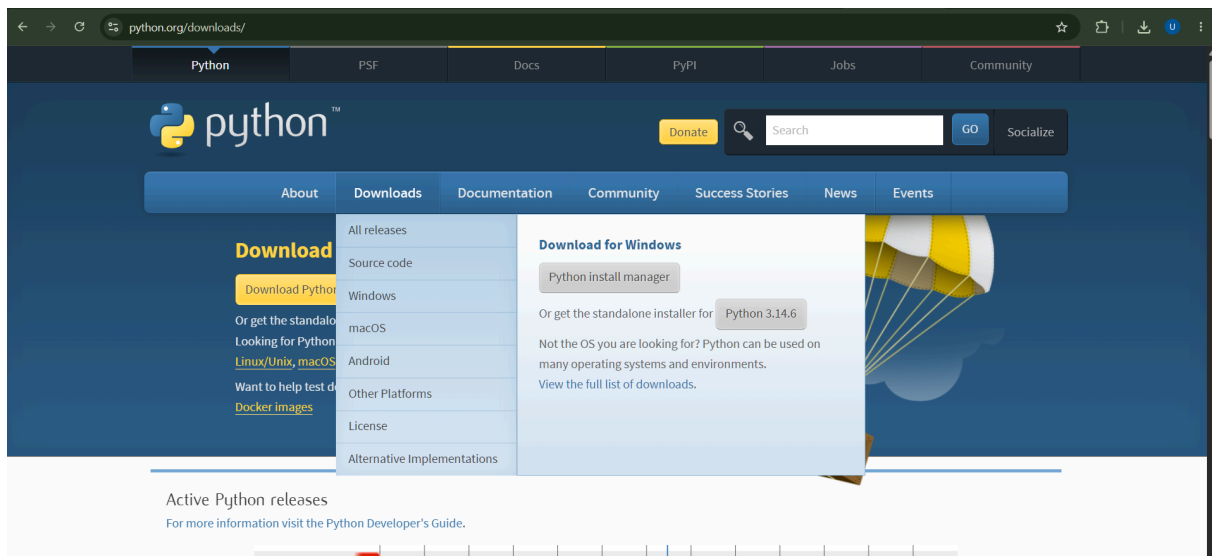
Download VS Code

<https://code.visualstudio.com/>

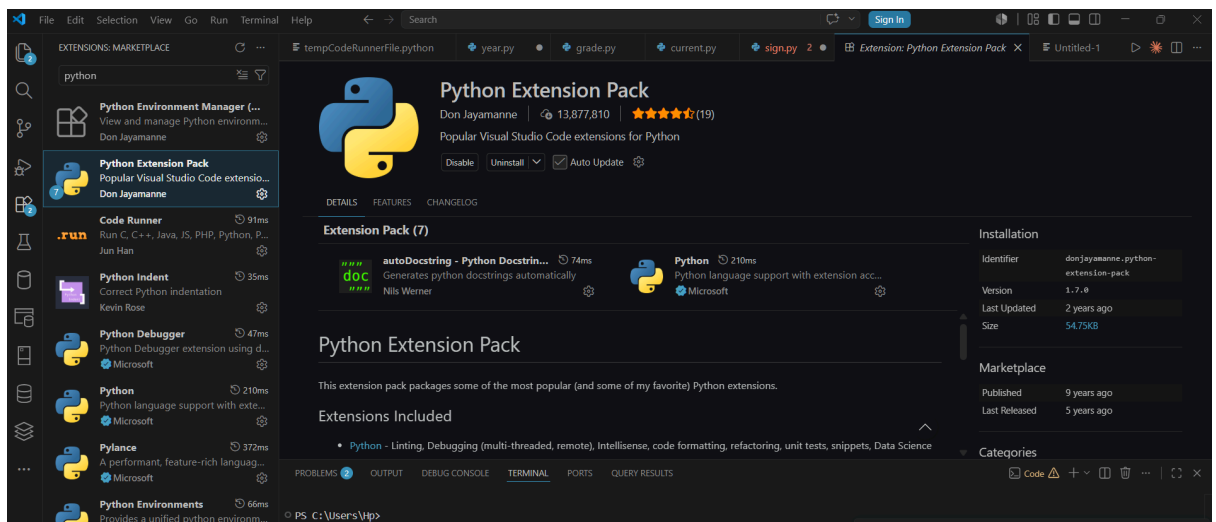


Install Python

<https://www.python.org/downloads/>



Install Python Extension



Result

```
1 print("hello Python")
```

```
PS C:\Users\Hp> python -u "
hello Python
```

10. Binary Format

Binary format represents data using only two digits, 0 and 1.

Example:

13

$13 \div 2 = 6$ remainder 1

$6 \div 2 = 3$ remainder 0

$3 \div 2 = 1$ remainder 1

$1 \div 2 = 0$ remainder 1

Binary Value : 1101

11. Why Programming Languages Are Needed

Humans cannot write everything in binary. Programming languages are used to communicate instructions to a computer in a form that humans can easily understand and write.

Specialized tools like compiler and interpreter then translate this human-readable code into the machine language the central processing unit (CPU) requires

12. High-Level Programming Languages

A High-Level Programming Language (HLL) is a programming language that is easy for humans to read, write, and understand.

Instead of writing binary code (0 and 1), programmers write instructions using English-like syntax.

Examples:

- Python
- Java
- C++
- JavaScript

Advantages:

- Easy to read
- Easy to write
- Faster development

13. Compiler vs Interpreter

A compiler translates the entire source code into machine code at once before execution, creating a standalone executable file.

Examples: C++ , C

An interpreter reads and translates the source code line by line in real-time as the program is running.

Example: Python , JavaScript

Feature	Compiler	Interpreter
What it does	Translates the entire code at once	Translates the code line-by-line
Speed (Execution)	Faster (after compiling)	Slower (because it translates while running)
Error checking	Shows all errors at once after compiling	Stops at the first error it finds
Output	Creates a separate file (machine code)	Does not create a separate file
Example Languages	C, C++, Java (compiled to bytecode)	Python, JavaScript, PHP

14. High-Level Code to Machine-Level Language

A **Language Translator** is a specific type of system software that acts as a converter.

It serves to connect high-level, human-readable code with the machine-level binary sequences consisting of 0s and 1s that a computer processor can execute.

High-Level Code → Compiler/Interpreter → Machine Code (0s and 1s) → CPU Execution → Output.

15. Python Variables

A variable in Python is like a container or label that holds a value. You can use it to store data like numbers, text etc., and then use that data later in your code.

```
▶ name = "SSP GOBAL"
  age = 3

  print(name)
  print(age)

... SSP GOBAL
   3
```

16. Python Data Types

In Python, a data type is a classification that tells the interpreter what kind of value a variable holds, and what kind of operations you can perform on that value.

```
▶ name = "SSP Global"      # str
  age = 24                 # int
  salary = 50000.5         # float
  status = True            # bool

  print(name)
  print(age)
  print(salary)
  print(status)

... SSP Global
   24
  50000.5
   True
```

17. Number Data Types

A Number is a data type used to represent numeric values. Python supports several types of numbers, and they're used for mathematical operations, counting, measuring, etc.

1. Integer (**int**)

Stores **whole numbers** (no decimal point).

Example:

```
▶ age = 22
  marks = 95
  temperature = -5

  print(age)
  print(type(age))

... 22
    <class 'int'>
```

2. Float (**float**)

Stores numbers with a **decimal point**.

Example:

```
salary = 25000.50
pi = 3.14159

print(salary)
print(type(salary))

25000.5
<class 'float'>
```

3. Complex (**complex**)

Used for mathematical and scientific calculations.

Format:

$a + bj$

where:

- **a** = Real part
- **b** = Imaginary part
- **j** = Square root of -1

Examples:

```
num = 2 + 3j

print(num)
print(type(num))

... (2+3j)
<class 'complex'>
```

18. Arithmetic Operations

```
a = 10
b = 3

print(a+b) #addition
print(a-b) # subtraction
print(a*b) # multiplication
print(a/b) # division
print(a//b) # Floor Division
print(a%b) # Modulus
print(a**b) # Power

13
7
30
3.3333333333333335
3
1
1000
```

- **Addition (+):** Adds two operands (e.g., $10 + 3 = 13$).
- **Subtraction (-):** Subtracts the second operand from the first (e.g., $10 - 3 = 7$).
- **Multiplication (*):** Multiplies two operands (e.g., $10 * 3 = 30$).
- **Division (/):** Divides the first operand by the second, always returning a float (e.g., $10 / 3 = 3.33$).
- **Floor Division (//):** Divides and rounds down to the nearest whole number (e.g., $10 // 3 = 3$).
- **Power (**):** Raises the first operand to the power of the second (e.g., $10 ** 3 = 1000$).

- **Modulus (%):** Returns the remainder of the division of the first operand by the second (e.g., $10 \% 3 = 1$).

19. Strings

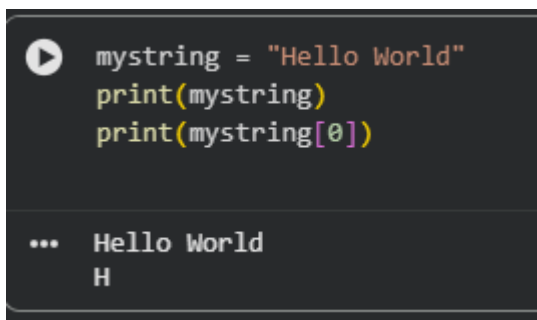
Strings: Strings are sequences of characters enclosed in single quotes (') or double quotes ("). Strings are one of the most commonly used data types in Python, and they provide a variety of methods to manipulate and interact with text-based data.

```
mystring = "Hello World"
```

```
print(mystring)
```

Explanation:

- You define a variable `mystring` and assign it the text "Hello World".
- `print(mystring)` simply outputs the entire string.



```
mystring = "Hello World"
print(mystring)
print(mystring[0])
```

... Hello World
H

Explanation:

`mystring[0]` accesses the first character of the string (Python starts counting from 0).



```
mystring = "Hello World"
print(mystring)
print(mystring[-1])
```

... Hello World
d

Explanation:

- `mystring[-1]` accesses the last character of the string.

- Negative indices count from the end (so -1 is the last character, -2 is second-last, etc.)

```
mystring = "Hello World"
print(mystring)
print(mystring[2 : 5])

Hello World
llo
```

Explanation:

- mystring is a string containing "Hello World".
- mystring[2:5] uses string slicing, which extracts a portion of the string.

How slicing works: string[start:stop]

- Start index: 2 → This is inclusive (it includes the character at index 2).
- Stop index: 5 → This is exclusive (it does not include the character at index 5).

```
mystring = "Hello World"
print(mystring[:])
```

It just returns the entire string unchanged.

```
mystring = "Hello World"
print(mystring[::2])

HlloWrld
```

This slices the string from beginning to end, taking every 2nd character.

mystring[::2] means:

- Start: from beginning
- Stop: until the end
- Step: 2 (every second character)

```
mystring = "Hello World"
print(mystring.upper())

... HELLO WORLD
```

mystring.upper()

- This converts all letters in the string to uppercase.
- "Hello World" becomes "HELLO WORLD"

```
mystring = "Hello World"
print(mystring.upper())

... HELLO WORLD
```

mystring.lower()

- This converts all letters to lowercase.
- "Hello World" becomes "hello world".

```
mystring = "Hello World"
print(mystring.split())

... ['Hello', 'World']
```

What .split() does:

- The .split() method breaks a string into a list of words, using spaces as the default separator.

```
mystring = "Hello World"
print(mystring.split('W'))

... ['Hello ', 'orld']
```

What's happening?

- You're using .split('W'), which tells Python:
 - "Split the string wherever you find the letter 'W'"

```

▶ mystring = "This is the Initial Text"
  print(mystring)
  modifiedstring = "This is the Initial Text {}".format("New Text Added")
  print(modifiedstring)
  print(mystring)

... This is the Initial Text
    This is the Initial Text New Text Added
    This is the Initial Text

```

mystring = "This is the Initial Text"

- You're assigning a string to the variable mystring.

modifiedstring = "This is the Initial Text {}".format("New Text Added")

- This uses .format() to insert "New Text Added" into the placeholder {}.

```

fruits = "apple"
print(fruits)
print("1: {} 2: {}".format("apple", "banana"))
print("1: {} 2: {}".format("apple", "banana"))

apple
1: apple 2: banana
1: apple 2: banana

```

- You're using .format() to insert values into {} placeholders.
- The first {} gets "apple", the second gets "banana".

```

fruits = "apple"
print("1: {a} 2: {b}".format(a="apple",b="banana"))

1: apple 2: banana

```

This code uses named placeholders with the .format() method. Inside the string "1: {a} 2: {b}", {a} and {b} are placeholders that get replaced by the values provided in .format(). Specifically, a="apple" and b="banana" assign "apple" to {a} and "banana" to {b}.

20. Lists

A List is a collection of multiple items stored in a single variable.

Lists are:

- Ordered (items have positions/indexes)
- Mutable (can be changed)
- Allow duplicate values
- Can store different data types

```
grocery_list = ["apples", "bananas", "carrots", "milk"]
```

```
print("Grocery List:", grocery_list)
```

```
# Accessing the first item
```

```
print("First item:", grocery_list[0])
```

```
# Adding an item to the list
```

```
grocery_list.append("eggs")
```

```
print("Updated List:", grocery_list)
```

```
# Removing an item from the list
```

```
grocery_list.remove("bananas")
```

```
print("After removing bananas:", grocery_list)
```

```
# Accessing a sublist (first two items)
```

```
print("Sublist:", grocery_list[:2])
```

This Python code demonstrates basic list operations on a `grocery_list`. Initially, the list contains `["apples", "bananas", "carrots", "milk"]`. The code performs the following actions:

1. Prints the entire list: `print("Grocery List:", grocery_list)` displays the original list.
2. Accesses the first item: `grocery_list[0]` retrieves the first item in the list, which is "apples", and prints it.
3. Adds a new item: `.append("eggs")` adds "eggs" to the end of the list, and the updated list is printed.
4. Removes an item: `.remove("bananas")` removes the first occurrence of "bananas", and the modified list (without bananas) is printed.
5. Accesses a sublist: `grocery_list[:2]` retrieves the first two items from the list, which are "apples" and "carrots", and prints this sublist.

Python

```
# Create a list of fruits
```

```
fruits = ["apple", "banana", "cherry", "orange", "kiwi"]
```

List Creation: A list of fruits is created with five items: "apple", "banana", "cherry", "orange", and "kiwi".

```
number_list = [1,2,3]
print(number_list)
print(len(number_list))

[1, 2, 3]
3
```

Creating and printing a list:

- `number_list = [1, 2, 3]` creates a list of integers.
- `print(number_list)` prints the list: `[1, 2, 3]`.

Getting the length of the list:

- `len(number_list)` returns the number of elements in the list `number_list`, which is 3 since the list contains three elements.
- `print(len(number_list))` prints the length of the list: 3.

```
fruits = ["apple", "banana", "cherry"]
more_fruits = ["orange", "kiwi", "grapes"]
fruits.extend(more_fruits)
print(fruits)

['apple', 'banana', 'cherry', 'orange', 'kiwi', 'grapes']
```

Creating two lists of fruits:

- `fruits = ["apple", "banana", "cherry"]` defines a list of fruit names.
- `more_fruits = ["orange", "kiwi", "grapes"]` defines another list of fruits.

Extending a list:

- `.extend(more_fruits)` is used to add all elements of the `more_fruits` list to the `fruits` list.
- After the extension, the `fruits` list becomes: `["apple", "banana", "cherry", "orange", "kiwi", "grapes"]`.

Printing the updated list:

- `print(fruits)` prints the extended fruits list: `["apple", "banana", "cherry", "orange", "kiwi", "grapes"]`.

```
# Remove and return the last item (by default)
last_fruit = fruits.pop()
print("Removed fruit:", last_fruit)
print("Updated list:", fruits)

Removed fruit: grapes
Updated list: ['apple', 'banana', 'cherry', 'orange', 'kiwi']
```

Removing the Last Item:

- The .pop() method removes the last item of the list by default, which is "kiwi".
- The list is updated to ["apple", "banana", "cherry", "orange"].

```
# Remove and return the fruit at index 1 (second item)
second_fruit = fruits.pop(1)
print("Removed second fruit:", second_fruit)
print("Updated list:", fruits)
```

```
Removed second fruit: banana
Updated list: ['apple', 'cherry', 'orange', 'kiwi']
```

Removing an Item by Index:

- The .pop(1) method removes the item at index 1 (which is "banana").
- The list is updated to ["apple", "cherry", "orange"].

```
# Create a list of fruits
fruits = ["apple", "banana", "cherry", "orange", "kiwi"]
fruits.reverse()
print(fruits)
```

```
['kiwi', 'orange', 'cherry', 'banana', 'apple']
```

Reversing the List

- fruits.reverse() reverses the order of the elements in the list fruits, so the last item becomes the first and vice versa.
- The reversed list is printed: ['kiwi', 'orange', 'cherry', 'banana', 'apple'].

```
number_list = [1,5,8,9,2,6]
number_list.sort()
print(number_list)
```

```
[1, 2, 5, 6, 8, 9]
```

Sorting the List:

- `number_list.sort()` sorts the list `number_list` in ascending order (numerically).
- The sorted list is printed: `[1, 2, 5, 6, 8, 9]`.

```
my_list = [1,2,3,['a','b','c']]
print(my_list)

[1, 2, 3, ['a', 'b', 'c']]
```

Displaying a Nested List:

- `my_list = [1, 2, 3, ['a', 'b', 'c']]` creates a list containing an internal sublist as its final element.
- `print(my_list)` outputs every component, demonstrating the nested structure: `[1, 2, 3, ['a', 'b', 'c']]`.

```
my_list = [1,2,3,['a','b','c']]
print(my_list[3])
print(my_list[3][1])

... ['a', 'b', 'c']
    b
```

Accessing elements within a sublist:

- `my_list[3]` retrieves the fourth element, which is the internal list `['a', 'b', 'c']`, and prints it.
- `my_list[3][1]` targets the second component inside the sublist, "b", and outputs its value.

21. Dictionary:

In Python, a dictionary is a classification that manages data through key-value pairs.

```
my_dict = {  
    "name": "Shiva",  
    "age": 34,  
    "city": "Tirupathi"  
}  
print(my_dict)  
  
{'name': 'Shiva', 'age': 34, 'city': 'Tirupathi'}
```

Explanation:

- A dictionary my_dict is created with three key-value pairs:
 - "name": "Shiva": The key is "name", and its value is "Shiva".
 - "age": 34: The key is "age", and its value is 34.
 - "city": "Tirupathi": The key is "city", and its value is "Tirupathi".

```
person = {  
    "name": "Alice",  
    "age": 30,  
    "address": {"city": "Wonderland", "zipcode": "12345"}  
}  
name = person["name"]  
age = person["age"]  
city = person["address"]["city"]  
print(name)  
print(age)  
print(city)  
  
Alice  
30  
Wonderland
```

Explanation:

- An initial dictionary named person is established with three specific key-value pairs:
 - "name": "Alice": This pair uses the key "name" to represent the data "Alice".
 - "age": 30: This pair uses the key "age" to represent the numeric data 30.

- "address": {"city": "Wonderland", "zipcode": "12345"}: Here, the key "address" points to a nested dictionary containing location details.

- Retrieving Data from the Dictionary:

- person["name"] is used to extract the value linked to "name", resulting in "Alice".
- person["age"] targets the "age" key to pull the value 30.
- person["address"]["city"] performs nested indexing to find "Wonderland" within the address sub-dictionary.

- Outputting Results:

- print(name) displays the string "Alice".
- print(age) displays the integer 30.
- print(city) displays the location "Wonderland".

```
items = {  
    "item1" : "Mangoes",  
    "item2" : "Potatoes",  
    "item3" : "curry leaves"  
}  
print ("Before modifying list")  
print(items)  
items["item3"] = "brinjal"  
print ("After modifying list")  
print(items)
```

```
Before modifying list  
{'item1': 'Mangoes', 'item2': 'Potatoes', 'item3': 'curry leaves'}  
After modifying list  
{'item1': 'Mangoes', 'item2': 'Potatoes', 'item3': 'brinjal'}
```

Explanation:

1. Creating the Dictionary:

- The dictionary *items* has three key-value pairs:
 - "item1": "Mangoes"

- "item2": "Potatoes"
- "item3": "curry leaves"

2. Printing the Dictionary Before Modification:

- `print("Before modifying list")` prints a message before showing the original dictionary.
- `print(items)` displays the dictionary as it is: `{'item1': 'Mangoes', 'item2': 'Potatoes', 'item3': 'curry leaves'}`.

3. Modifying the Value of a Key:

- `items["item3"] = "brinjal"` updates the value associated with "item3" from "curry leaves" to "brinjal".

4. Printing the Dictionary After Modification:

- `print("After modifying list")` prints a message before showing the updated dictionary.
- `print(items)` displays the modified dictionary: `{'item1': 'Mangoes', 'item2': 'Potatoes', 'item3': 'brinjal'}`.

22. Tuples

```
mytuple = (1,2,3)
print(mytuple[1])

# Convert the tuple to a list to allow modification
my_list = list(mytuple)
my_list[0] = "5"

# Convert it back to a tuple if desired, or keep as a list
mytuple = tuple(my_list)

print(mytuple)

2
('5', 2, 3)
```

This code demonstrates how tuples work in Python. In the first part, a tuple `mytuple = (1, 2, 3)` is created, and `mytuple[1]` accesses the second element (value 2), which is valid and prints correctly. However, in the second part, after printing the second element again, the code attempts to modify the first

element with `mytuple[0] = "5"`. This causes a `TypeError` because tuples are immutable in Python, meaning their elements cannot be changed after creation. Therefore, the code will crash at that line with an error message stating that tuple objects do not support item assignment.

23. Sets

```
x = set()
x.add(1)
x.add(2)
x.add(2)
x.add(5)
print(x)
tags = {"Python", "Programming", "Tech ", "Programming", }
print(tags)
```



```
{1, 2, 5}
{'Python', 'Programming', 'Tech '}
```

This code demonstrates how sets work in Python, focusing on their properties of uniqueness and unordered storage. Here's the explanation:

1. 1. Creating and Modifying a Set:

- `x = set()` creates an empty set.
- `x.add(1)`, `x.add(2)`, and `x.add(5)` add elements to the set.
- `x.add(2)` is repeated, but sets do not allow duplicates, so 2 is stored only once.
- Final output of `print(x)` will show: `{1, 2, 5}` (order may vary because sets are unordered).

2. 2. Defining a Set with Duplicate Values:

- `tags = {"Python", "Programming", "Tech ", "Programming"}` creates a set with four items, but "Programming" is listed twice.
- Sets automatically remove duplicates, so "Programming" appears only once.
- The string "Tech " has a trailing space, so it's considered distinct from "Tech" (no trimming happens automatically).

- Output of `print(tags)` will be something like: `{'Python', 'Tech ', 'Programming'}` (again, order is not guaranteed).

```
is_active = True
is_completed = False
print(is_completed)
print(f"Is user active or not : {is_active}")

False
Is user active or not : True
```

24. Boolean Variables

Explanation:

1. Boolean Variables:

- `is_active = True`: A variable `is_active` is assigned the Boolean value `True`.
- `is_completed = False`: Another variable `is_completed` is assigned `False`.

2. Printing a Boolean:

- `print(is_completed)` simply prints the value of `is_completed`, which is `False`.

3. Formatted String (f-string):

- `print(f"Is user active or not : {is_active}")` uses an f-string to include the value of `is_active` within the printed message.

25. Comparison Operators:

In programming, comparison operators are specific symbols or keywords utilized to evaluate the relationship between two values. These operations always result in a Boolean outcome, yielding either **True** or **False** based on the validity of the comparison.

<i>op</i>	<i>meaning</i>	True	False
<code>==</code>	<i>equal</i>	<code>2 == 2</code>	<code>2 == 3</code>
<code>!=</code>	<i>not equal</i>	<code>3 != 2</code>	<code>2 != 2</code>
<code><</code>	<i>less than</i>	<code>2 < 13</code>	<code>2 < 2</code>
<code><=</code>	<i>less than or equal</i>	<code>2 <= 2</code>	<code>3 <= 2</code>
<code>></code>	<i>greater than</i>	<code>13 > 2</code>	<code>2 > 13</code>
<code>>=</code>	<i>greater than or equal</i>	<code>3 >= 2</code>	<code>2 >= 3</code>

Comparisons with `int` operands and a `bool` result

Example:

```
a = 10
b = 20
print(a == b)  # False
print(a != b)  # True
print(a < b)   # True
print(a >= 5)  # True

False
True
True
True
```

Comparison operators are used to compare two values, and they always return a Boolean result—either True or False.

The examples likely demonstrate how these operators check the relationship between values:

- **Equal to (`==`):** Checks if two values are the same (e.g., `10 == 20` is False).
- **Not equal to (`!=`):** Checks if two values are different (e.g., `10 != 20` is True).
- **Greater than (`>`) / Less than (`<`):** Compares magnitude (e.g., `10 < 20` is True).

- **Greater than or equal to ($\geq$) / Less than or equal to ($\leq$):** Checks if a value is greater/less than or exactly equal to another (e.g., $10 \geq 5$ is True).

26. Logical operators:

Logical operators are used to combine multiple Boolean expressions (conditions) and return a True or False result based on logical relationships.

Operator	Description	Example	Result
and	Returns True if both conditions are True.	$x > 3$ and $y < 15$	True if both are True
or	Returns True if at least one condition is True.	$x > 3$ or $y < 10$	True if at least one is True
not	Negates the condition. Returns True if the condition is False, and False if the condition is True.	not $x > 10$	True if the condition is False, False otherwise

Example:

```
x = 10
y = 5
print(x > 5 and y < 10)    # True (both are True)
print(x < 5 or y < 10)     # True (only one needs to be True)
print(not(x == y))
# True (x is not equal to y)

True
True
True
```

There are three primary logical operators:

1. **Logical AND:** This operator yields a **True** outcome only when *both* conditions are satisfied.
 - a. *Example:* The expression $(10 > 5 \text{ and } 5 < 10)$ results in **True**.
2. **Logical OR:** This operator yields **True** if *at least one* of the evaluated conditions is true.
 - a. *Example:* The expression $(10 < 5 \text{ or } 5 < 10)$ results in **True**.

3. **Logical NOT:** This operator reverses the Boolean value, returning **True** for false conditions and **False** for true ones.
 - a. *Example:* The expression (not (10 == 5)) results in **True**.

27. Condition Statements

if Statement

Executes a block of code only if the condition is true.

```
x = 10
if x > 5:
    print("x is greater than 5")

x is greater than 5
```

if...else Statement

Lets you run one block if the condition is true, and another if it's false.

```
x = 3
if x > 5:
    print("x is greater than 5")
else:
    print("x is not greater than 5")

x is not greater than 5
```

Explanation:

- The if part checks a condition (a question like: "Is x greater than 5?").
- If the answer is yes (True), Python runs the code inside the if block.
- If the answer is no (False), Python skips the if block and goes to the else block.
- The else block is like a backup plan — it runs only if the if condition is False.

if...elif Statement:

It's a way in Python to check multiple conditions, one after the other.

- if = checks the first condition.
- elif = checks the next condition only if the first one is False.
- You can have as many elif statements as you want.

```
age = 15
if age < 13:
    print("Child")
elif age < 18:
    print("Teen")
else:
    print("Adult")
Teen
```

- When the age variable is below 13, the output displays "Child".
- For ages between 13 and 17 inclusive, the program outputs "Teen".
- In every other case (18 or above), the system prints "Adult".

28. For Loops:

A for loop is a control flow statement in programming that is used to iterate over (or loop through) a sequence (like a list, string, or range). It helps you perform repetitive tasks efficiently by executing a block of code multiple times.

```
# List of numbers
numbers = [1, 2, 3, 4, 5]
# Using a for loop to print each number in the list
for num in numbers:
    print(num)

1
2
3
4
5
```

Numerical Data Collection:

An initial sequence of integers is established as follows: [1, 2, 3, 4, 5].

Looping Mechanism:

The logic utilizes a for loop to traverse every individual component within the sequence. In each cycle, the num variable serves as a container for the specific item currently being processed.

Process Flow(Execution):

- Initially, the variable num points to 1 (the starting element) and outputs its value.
- Moving forward, num shifts to the second entry, 2, and displays it.
- This repetitive action proceeds until every digit in the entire sequence has been outputted.

29.Tuple Unpacking

```
my_list = [(1, 2), (3, 4), (5, 6)]
for items in my_list:
    print(items)

(1, 2)
(3, 4)
(5, 6)
```

This line creates a list called my_list. The list contains three elements, and each element is a tuple — which is an immutable sequence.

```
#Tuple Unpacking
my_list = [(1, 2), (3, 4), (5, 6)]
for (t1,t2) in my_list:
    print(t1)
```

```
1
3
5
```

Rather than mapping the whole tuple into a single variable, this approach unpacks the internal values directly into two distinct variables, `t1` and `t2`.

Execution Breakdown:

- Initial cycle: The pair (1, 2) is assigned to (t1, t2), resulting in 1 being outputted.
- Following cycle: The pair (3, 4) is mapped to (t1, t2), resulting in 3 being outputted.
- Final cycle: The pair (5, 6) is mapped to (t1, t2), resulting in 5 being outputted.

30. String Formatting Methods

Showing two different ways to print the value of a variable in Python — both do the same thing, but with different styles. Let's break it down:

```
a = 5
print("Value of a: {}".format(a))

Value of a: 5
```

In the string "Value of a: {}", the curly braces function as a format string.

The {} acts as a placeholder, which Python replaces with the specific data held in variable `a`.

By calling `.format(a)`, the system populates the placeholder with the integer 5.

```
a = 5
print("Value of a: " + str(a))
Value of a: 5
```

In the provided code, the sequence "Value of a: " functions as a literal text string.

The `str(a)` function is used to transform the numerical integer data into a textual representation, "5".

The addition symbol (+) acts as a concatenation tool to merge these separate strings into one.

```
x = [1,2,3,4]
output = []
for i in x:
    output.append(i**2)
print(output)
[1, 4, 9, 16]
```

Initial Data Sequence:

The process begins with a sequence named `x`, established with four distinct integers.

Iteration Mechanism:

A for loop is utilized to traverse every individual component, represented by variable `i`, within the sequence `x`.

Process Flow (Execution):

During each cycle, the logic calculates the power of `i` squared (`i**2`) and stores the result in a separate output collection as follows:

- Initial cycle: `i` is 1, resulting in a value of 1.
- Second cycle: `i` is 2, resulting in a value of 4.
- Third cycle: `i` is 3, resulting in a value of 9.

- Final cycle: i is 4, resulting in a value of 16.

31. List Comprehension

```
x = [1,2,3,4]
output = [i**2 for i in x]
print(output)

[1, 4, 9, 16]
```

The code uses list comprehension to create a new list in a concise and efficient way.

The original list x contains the integers: [1, 2, 3, 4].

The expression [i**2 for i in x]:

- Loops through each element i in the list x.
- Computes i**2, which means squaring each number.
- Collects all the squared values into a new list called output.

The print(output) statement displays: [1, 4, 9, 16].

This method is:

- Shorter than a traditional for loop.
- More readable and Pythonic.
- A common and efficient way to transform or filter lists in Python.

32.While Loops

A while **loop** in Python is used to repeatedly execute a block of code as long as a specified condition remains TRUE

1. Syntax of a while Loop

```
while condition:
    # code to execute while the condition is True.
    # update the condition variable
```

2. A simple Example

```
count = 1

while count <= 3:
    print(f"Count is: {count}")
    count += 1 # Increments count by 1 so the loop doesn't run forever

print("Loop finished!")

Count is: 1
Count is: 2
Count is: 3
Loop finished!
```

3. Infinite Loop

```
# WARNING: This is an infinite loop!
count = 1
while count <= 3:
    print(count)
    # Missing 'count += 1', so count stays 1 forever

1
1
1
1
```

If the condition inside a while loop never becomes false , the loop will run forever, freezing your program. This is called an **infinite loop**.

4. Controlling Loop

a. Break


```
# Stop the loop if we find the number 3
num = 1
while num <= 5:
    if num == 3:
        break
    print(num)
    num += 1
```

```
1
2
```

break immediately terminates the loop completely, skipping any remaining code inside it.

b. continue

```
# Skip printing the number 3
num = 0
while num < 5:
    num += 1
    if num == 3:
        continue
    print(num)
```

```
1
2
4
5
```

continue skips the rest of the current iteration and jumps straight back to evaluating the condition for the next turn.

33. Functions:

A function in programming is a block of reusable code designed to perform a specific task. Functions allow you to group related code together, so you can call that code whenever needed, which makes your code more organized, reusable, and easier to understand.

```
def greet():  
    print("Welcome to SSP")  
greet()  
  
Welcome to SSP
```

Explanation:

Defining the Function (def greet()):

This syntax establishes a new function identified as greet.

In programming, these are modular segments of logic that remain dormant until triggered.

The Function Body:

The instruction `print("Welcome to SSP")` constitutes the internal code logic.

Execution of the function results in displaying the greeting "Welcome to SSP".

Invoking the Function (greet()):

This statement represents a formal function call.

It instructs the Python interpreter to run the sequence defined within `greet`.

```
def greet():  
    """  
    This is Informational Text  
    """  
    print("Welcome to SSP")  
greet()  
  
Welcome to SSP
```

def greet():

- This syntax establishes a new function identified as greet.

""" This is Informational Text """

- This represents a docstring, which is short for documentation string.
- It functions as a unique multi-line comment meant to explain the purpose of the logic.
- It serves for better documentation and help tools without changing the execution results.

print("Welcome to SSP")

- The instruction constitutes the internal logic to display the greeting message.

greet()

- This statement represents a formal function call to execute the code.

```
#Will not print anything
def greet():
    return "Welcome to SSP"
greet()
```

def greet():

- This syntax establishes a new function identified as greet.

return "Welcome to SSP"

- The logic passes the greeting back to the calling statement rather than outputting it.

- This sends the result back to the invoker without an automatic display to the user.

greet()

- The function is formally invoked, but the sent back value remains unhandled.

- Consequently, the execution yields no visible text on the console..

```
# Parameters
def add(a, b):
    # Function definition with parameters a and b
    result = a + b      # Adds the two parameters
    print(result)
    # Prints the result
add(10, 1)
    # Calls the function with arguments 10 and 1
```

Function Definition

- The syntax `def add(a, b):` establishes a new function identified as `add`, designed to accept two distinct inputs, `a` and `b`.

Performing Addition

- Within the internal logic, the instruction `result = a + b` combines the values provided for both parameters.

Storing the Result

- This calculated sum is then placed into a container variable named `result` for subsequent use.

Printing the Result

- The `print(result)` statement executes to display the numeric data held in the `result` onto the console.

Calling the Function

- Invoking `add(10, 1)` triggers the logic, passing the integer 10 to `a` and 1 to `b`.

What Happens During Execution

- The processor calculates the sum of 10 and 1, yields the value 11, and outputs the result to the user.

```
def add(a, b):  
    if type(a) == type(b) == type(11):  
        print("Added Result: {}".format(a + b))  
    else:  
        print("Please provide integer numbers")  
add(10, 1)  
  
Added Result: 11
```

Function Definition

- The syntax `def add(a, b) :` establishes a new function identified as `add`, designed to accept two distinct inputs, `a` and `b`.

Type Verification

- Within the internal logic, the system evaluates if both parameters are integers by comparing their data types against the integer `11`.

Conditional Execution

- When the validation is successful, the processor calculates the sum and utilizes a formatted string to display "Added Result: " along with the total.

Error Handling

- Should either input fail the integer check, the `else` block triggers to output the message "Please provide integer numbers".

Invoking the Logic

- Calling `add(10, 1)` provides two whole numbers, ensuring the condition is satisfied for execution.

Execution Breakdown

- The processor combines 10 and 1, yields a numeric result of 11, and displays the final string "Added Result: 11" to the user.

35. Scope of Variables

The LEGB rule in Python describes the search order for variable names when they are referenced in the code. LEGB stands for:

1. L: Local
2. E: Enclosing Function Locals
3. G: Global
4. B: Built-in

Local scope: Refers to variables defined within the current function or block.

Enclosing scope: Refers to variables in any enclosing functions (functions that contain the current function).

Global scope: Refers to variables defined at the top level of the module or script.

Built-in scope: Refers to Python's standard library functions, exceptions, and built-in objects (e.g., `print()`, `len()`).

```
name = "This is Global Name"
def invite():
    name = "Shiva"
def subinvite():
    print ("Welcome " +name)
invite()
```

Global Variable:

- The name variable is globally defined with the value "This is Global Name".
- This global variable is accessible from anywhere in the code unless shadowed by a local variable.

invite() Function:

- Inside the invite() function, a local variable name = "Shiva" is defined.
- This local name variable shadows or overrides the global name within the invite() function.
- The name defined in invite() is only available within that function's scope.

Nested subinvite() Function:

- The function subinvite() is defined inside invite(), and it tries to print a message using the name variable.
- Since subinvite() is inside invite(), it looks for the name variable in the enclosing invite() function's scope. It does not check the global scope unless there's no name in the local scope.

- The name variable in invite() has the value "Shiva", so subinvite() prints "Welcome Shiva".

Calling subinvite() in invite():

- Inside the invite() function, subinvite() is called.
- The call to subinvite() triggers the print statement, using the name variable from invite()'s scope.

Calling invite():

- Finally, the invite() function is called, which calls subinvite() and results in the print output "Welcome Shiva".

```
name = "This is Global Name"
def invite():
    name = "Shiva"
def subinvite():
    print ("Welcome " +name)
subinvite() #Prints local
invite()

Welcome This is Global Name
```

Global Variable:

- name = "This is Global Name" is defined globally, accessible throughout the program unless overshadowed by a local variable.

Local Variable Inside invite() Function:

- Inside the invite() function, a local variable name = "Shiva" is defined.

- This local variable shadows the global name within the invite() function scope.

Nested Function subinvite():

- The subinvite() function is defined inside the invite() function.
- Inside subinvite(), the code tries to access and print the value of name.

Lexical Scoping:

- subinvite() first looks for name in its local scope (it doesn't find it there).
- It then checks the enclosing scope (invite()), where name = "Shiva" is found.
- The local name (inside invite()) is used, not the global name.

Calling the invite() Function:

- When invite() is called, it calls subinvite(), and subinvite() prints Welcome Shiva.

```
class Banking:

    def __init__(self, tbal=1000):

        self.tbal = tbal

    def deposit(self, amount):

        self.tbal += amount

        print(f"Deposited Amount: {amount}")

        print(f"Total balance after depositing amount {amount} is {self.tbal}")

    def withdraw(self, amount):

        if amount > self.tbal:

            print("\nInsufficient balance! Withdrawal cancelled.")

        else:

            self.tbal -= amount

            print(f"Withdrawn Amount: {amount}")

            print(f"Total balance after withdrawal is {self.tbal}")

    def check_balance(self):

        print(f"\nTotal Balance in the account is: {self.tbal}")

# ---- User Interaction ----
```

```

print("""Please enter the operation to be performed:

1. Deposit
2. Withdraw
3. Check Available Balance

""")

bank = Banking()

useroption = int(input("Enter your choice (1/2/3): "))

if useroption == 1:

    amount = int(input("Enter the amount to be deposited: "))

    bank.deposit(amount)

elif useroption == 2:

    amount = int(input("Enter the amount to be withdrawn: "))

    bank.withdraw(amount)

elif useroption == 3:

    bank.check_balance()

else:

    print("\nInvalid option. Please choose 1, 2, or 3.")

```

35. Lambda Functions

A **lambda function** in Python is a small, anonymous function. "Anonymous" means that the function is defined without a name and without using the standard `def` keyword.

1. Syntax of a Lambda Function

lambda arguments: expression

lambda: The keyword that tells Python you are creating an anonymous function.

arguments: A comma-separated list of inputs (just like parameters in a normal function).

expression: A single line of code that gets executed and automatically returned.

2. Common uses

a. With filter():

The filter () function extracts elements from an iterable if they meet a specific condition.

```
numbers = [1, 2, 3, 4, 5, 6]

# Filter out only the even numbers
even_numbers = list(filter(lambda x: x % 2 == 0, numbers))

print(even_numbers) # Output: [2, 4, 6]
```

[2, 4, 6]

b. With map():

The map() function applies a specific operation to every item in an iterable.

```
numbers = [1, 2, 3, 4]

# Square every number in the list
squared_numbers = list(map(lambda x: x**2, numbers))

print(squared_numbers) # Output: [1, 4, 9, 16]

[1, 4, 9, 16]
```

c. with sorted():

You can use a lambda function to customize how a collection of data is sorted.

```
# A list of tuples containing (name, age)
students = [("Shiva", 25), ("Sai", 22), ("Arun", 28)]

# Sort the students based on their age (the element at index 1)
sorted_by_age = sorted(students, key=lambda student: student[1])

print(sorted_by_age)
# Output: [('Sai', 22), ('Shiva', 25), ('Arun', 28)]

[('Sai', 22), ('Shiva', 25), ('Arun', 28)]
```

36. Object-Oriented Programming (OOP)

OOPS stands for:

Object Oriented Programming System

It is a programming method used to organize code in a better and smarter way.

OOPS helps programmers:

- write clean code
- reuse code
- manage large projects easily
- reduce repetition

Real Life Understanding of OOPS

Consider the analogy of a vehicle, specifically a Car.

Such a machine comprises:

Distinct Properties:

- Its visual color
- The manufacturer brand
- The current speed

Operational Actions:

- To start the ignition
- To stop the engine
- To move forward

Within the Python environment, we represent these real-world entities through OOPS principles.

37. Main Concepts in OOPS

For beginners, first understand:

- Class
- Object

These are the foundation of OOPS.

What is a Class?

A Class is a blueprint or design for creating objects. **OR** A Class is a template that defines properties and actions.

Simple Real Life Example

Consider the analogy of a handheld device, specifically a Mobile Phone.

Every such device comprises:

Distinct Properties:

- The manufacturer brand
- Its visual color

- The hardware RAM capacity

Operational Actions:

- To perform voice calling
- To utilize the camera
- To engage in messaging

Within this framework, the design or template is identified as a Class, while the actual physical devices are known as Objects.

Definition of Class

A Class is a collection of:

- Variables → Properties/Data
- Functions → Actions/Behaviors

Example Explanation

Within this framework, a Student Class might comprise:

Distinct Properties:

- Its identifying name
- The unique roll number

- The academic marks data

Operational Actions:

- To perform reading
- To engage in writing
- To participate in examinations

What is an Object?

An Object is a real instance created from a class. **OR** Object is the actual thing created using the class design.

Real Life Example

Class → Bike Design

Object → Honda Bike, Yamaha Bike

Class → Student Form

Object → Ravi, Sita, Ram

Why Do We Use Classes?

Without class:

- code becomes lengthy

- repeated code increases
- difficult to manage projects

With class:

- ✓ code becomes reusable
- ✓ easy to manage
- ✓ organized structure
- ✓ faster development

38. Constructors (*init*)

a **constructor** is a special type of method that is automatically called when a new object (instance) of a class is created.

1. `__init__` Method

In Python, the constructor is always named `__init__` (with two underscores before and after the word "init", which stands for initialization).

```
class ClassName:
    def __init__(self, parameter1, parameter2):
        self.attribute1 = parameter1
        self.attribute2 = parameter2
```

2. Types of Constructors

A. Default Constructor

an initialization method that **takes no arguments except self**

```
class EmptyClass:
    # Python automatically gives this a default constructor behind the scenes
    pass

obj = EmptyClass()
```

B. Parameterized Constructor

A constructor that accepts parameters (arguments) along with **self** to dynamically initialize attributes

```
class Laptop:
    # Parameterized constructor
    def __init__(self, brand, processor, ram_gb):
        self.brand = brand          # Instance attribute
        self.processor = processor  # Instance attribute
        self.ram = ram_gb          # Instance attribute

    def display_specs(self):
        print(f"Brand: {self.brand}, CPU: {self.processor}, RAM: {self.ram}GB")

# Instantiating objects with unique initial values
laptop1 = Laptop("Apple", "M3", 16)
laptop2 = Laptop("Dell", "Intel i7", 32)

laptop1.display_specs() # Output: Brand: Apple, CPU: M3, RAM: 16GB
laptop2.display_specs() # Output: Brand: Dell, CPU: Intel i7, RAM: 32GB

Brand: Apple, CPU: M3, RAM: 16GB
Brand: Dell, CPU: Intel i7, RAM: 32GB
```

39. Inheritance

Within the OOPS framework, inheritance functions as a primary pillar. It enables a newly created **Child** or **Derived** class to acquire the properties and actions of a pre-existing **Parent** or **Base** class design.

1. Syntax

```
class ParentClass:
    # Attributes and methods of the parent
    pass

class ChildClass(ParentClass):
    # Inherits everything from ParentClass automatically
    # Plus, you can add new attributes and methods here
    pass
```

2. super() Function

The super () function is a built-in tool that allows the child class to call methods from its parent class.

```
class Animal:
    def sound(self):
        print("Animal makes sound")

class Dog(Animal):
    def sound(self):
        super().sound()
        print("Dog barks")

d = Dog()
d.sound()

Animal makes sound
Dog barks
```

3. Method Overriding

Method Overriding occurs when a child class provides its own implementation of a method that already exists in the parent class.

```
class Animal:
    def sound(self):
        print("Animal makes a sound")
class Dog(Animal):
    def sound(self):
        print("Dog barks")

d = Dog()
d.sound()

Dog barks
```

4. Types of Inheritances

Single Inheritance: One child inherits from one parent. (e.g., Parent → Child).

Multiple Inheritance: One child inherits from multiple parents. (e.g., Child → (father, Mother)).

Multilevel Inheritance: Inheritance happens in levels. (e.g., Grandparent → Parent → Child).

Hierarchical Inheritance: Multiple child classes inherit from a single parent class (e.g., Parent → (son, daughter)).

Hybrid Inheritance: Combination of two or more inheritance types.

```
# Base Class
class GrandParent:
    def method_gp(self):
        print("This is GrandParent")

# Single Inheritance
class Parent1(GrandParent):
    def method_p1(self):
        print("This is Parent1")

class Parent2:
    def method_p2(self):
        print("This is Parent2")

# Multiple Inheritance
class Child(Parent1, Parent2):
    def method_c(self):
        print("This is Child")

# Create object
obj = Child()
obj.method_gp() # Inherited from GrandParent
obj.method_p1() # Inherited from Parent1
obj.method_p2() # Inherited from Parent2
obj.method_c()  # Defined in Child

This is GrandParent
This is Parent1
This is Parent2
This is Child
```

40. Polymorphism

The word itself comes from Greek, meaning "**many forms.**" In programming, polymorphism allows different classes to have methods with the exact same name but different behaviors.

1. Method Overriding (Runtime Polymorphism)

Method overriding occurs when a child class provides a specific implementation for a method that is already defined in its parent class.

```
class Animal:
    def sound(self):
        print("Animal makes sound")
class Dog(Animal):
    def sound(self):
        print("Dog barks")

class Cat(Animal):
    def sound(self):
        print("Cat meows")
d = Dog()
c = Cat()

d.sound()
c.sound()

Dog barks
Cat meows
```

2. Compile-Time Polymorphism

It is also known as Method Overloading Same method name different parameters.

Note: Python does not support traditional method overloading like Java/C++.

3. Duck Typing

objects do not even need to inherit from the same parent class.


```

class Car:
    def start(self):
        print("Vroom! Engine ignited.")

class Laptop:
    def start(self):
        print("Bootng up the operating system...")

# This function doesn't care about the type/class of the object,
# it only cares if the object has a 'start' method.
def activate_device(device):
    device.start()

# Both work flawlessly despite being completely unrelated classes
activate_device(Car())      # Output: Vroom! Engine ignited.
activate_device(Laptop())   # Output: Bootng up the operating system...

Vroom! Engine ignited.
Bootng up the operating system...

```

41. Encapsulation

Encapsulation is the process of hiding data and allowing access to it only through methods.

Methods:

1. **Getters:** Used to **read/access** private data.

```

def get_name(self):
    return self.__name

```

2. **Setters:** Used to modify/update private data.

```

def set_name(self, name):
    self.__name = name

```

```

class Student:
    def __init__(self):
        self.__name = "Ramesh"    # Private Variable

    def get_name(self):            # Getter
        return self.__name

    def set_name(self, name):      # Setter
        self.__name = name

s = Student()

print(s.get_name())

s.set_name("Rahul")

print(s.get_name())

Ramesh
Rahul

```

42. Exception Handling

Exception Handling is a mechanism used to handle runtime errors so that the program does not crash.

1. try and except:

Python uses a try - except block to catch and handle exceptions.

- **try block:** Contains the code that *might* throw an error.
- **except block:** Contains the code that executes *only if* an error happens inside the try block **or** Handles an error

```

try:
    numerator = 10
    denominator = 0
    result = numerator / denominator # This raises a ZeroDivisionError
    print(result)
except ZeroDivisionError:
    # This runs instead of crashing the program
    print("Error: You cannot divide a number by zero!")

print("The program continues to run smoothly...")

```

```

Error: You cannot divide a number by zero!
The program continues to run smoothly...

```

2. Catching Multiple Specific Exceptions

A single try block can have multiple except blocks to handle different types of errors uniquely. It is always best practice to catch specific exceptions rather than using a generic except:

```

try:
    # Scenario 1: Index out of range
    my_list = [10, 20]
    print(my_list[5])

    # Scenario 2: Converting bad text to integer
    num = int("Hello")

except IndexError:
    print("Error: Looked for an index that doesn't exist in the list.")
except ValueError:
    print("Error: Cannot convert that string into a number.")

Error: Looked for an index that doesn't exist in the list.

```

3. else and finally

else: Runs if no exception occurs.

finally : Runs whether an exception occurs or not.

Block	When does it execute?	Common Use Case
else	Only if no exceptions were raised in the try block.	Code that should run only if the try block was 100% successful.
finally	Always, regardless of whether an error occurred or not.	Cleanup actions (e.g., closing a database connection or saving a file).

```

try:
    file = open("notes.txt", "r")
    content = file.read()
except FileNotFoundError:
    print("Error: The file could not be found.")
else:
    print("File read successfully!")
    print(content)
finally:
    print("Closing any open resources...")
    # This line runs no matter what happens above

```

```

Error: The file could not be found.
Closing any open resources...

```

4. raise Exception (optional)

raise is used to manually generate (throw) an exception when a specific condition occurs.

```
password = "abc"

if len(password) < 8:
    raise ValueError("Password must contain at least 8 characters")

-----
ValueError                                Traceback (most recent call last)
/tmp/ipykernel_1851/3340228125.py in <cell line: 0>()
      2
      3 if len(password) < 8:
----> 4     raise ValueError("Password must contain at least 8 characters")

ValueError: Password must contain at least 8 characters

-----
Next steps: Explain error
```

Common Exceptions

Exception	Cause
ZeroDivisionError	Dividing by zero
ValueError	Invalid input
TypeError	Wrong data type
IndexError	Invalid list index
KeyError	Invalid dictionary key
FileNotFoundError	File doesn't exist

43. File Handling

File Handling is the process of creating, reading, writing, and updating files in Python.

File Modes

Mode	Meaning
r	Read
w	Write (overwrites file)
a	Append
x	Create new file
r+	Read + Write
w+	Write + Read

44. Modules and Packages

Module : A module is a Python file containing reusable code such as functions, classes, and variables. (e.g., [calculator.py](#), main.py)

Package : A package is a directory that contains multiple modules and helps organize related code (e.g., MyPackage -> __init__.py , [calculator.py](#), [student.py](#), [employee.py](#))

45. Python Libraries

A Python Library is a collection of modules that provide pre-written code to perform specific tasks.

Why Use Libraries?

Without libraries, you write everything from scratch.

```
import math

print(math.sqrt(25))

5.0
```

The library already provides the functionality.

Common Python Libraries

1. Math Library

Used for mathematical operations.

```
import math

print(math.sqrt(16))
print(math.factorial(5))

4.0
120
```

2. Random Library

Used for random numbers.

```
import random

print(random.randint(1,10))

6
```

3. Datetime Library

Used for date and time.

```
import datetime

print(datetime.datetime.now())

2026-06-25 09:43:47.826514
```

4. OS Library

Used for interacting with the operating system.

```
import os

print(os.getcwd())

/content
```

5. NumPy

Used for numerical computations.

```
import numpy as np

arr = np.array([1,2,3])

print(arr)

[1 2 3]
```

6. Pandas

Used for data analysis.


```
import pandas as pd

data = {
    "Name": ["Ramesh"],
    "Age": [22]
}

df = pd.DataFrame(data)

print(df)
```

	Name	Age
0	Ramesh	22

7. Matplotlib

Used for graphs and charts.

```
import matplotlib.pyplot as plt

x = [1,2,3]
y = [10,20,30]

plt.plot(x,y)
plt.show()
```

